Architectural Justification

Clinical Trial Subject Recruitment Engine

Architecture Selection: We chose Layered Architecture for Clinical Trial Subject Recruitment Engine System

Approach

The system is structured into three main tiers. There's a Presentation Layer for the user interfaces (both the candidate portal and the researcher portal). Under that, a Business Layer handles the core logic with the Matching Engine and the Notification & Consent Service. Finally, a Data Layer manages the Trial and Candidate Database.

Rationale for the Choice

1. To Keep Things Manageable and Maintainable

There are two distinct sides to this system: candidates signing up and researchers managing trials. However, both workflows ultimately feed into the same central process—matching a person to a study. Using layers lets us keep that core matching logic in one clear place within the Business Layer. For a system of this size, it's a lot more straightforward to build, test, and update than if we had split it into separate, independently deployed microservices. It keeps the codebase cohesive.

2. For Consistent and Reliable Data Governance

Strict control over how data is handled is required, especially for trial protocols and candidate information. By routing every single read or write operation through a single Data Layer, we create a single point of control. This means we can enforce eligibility rules, input validation, and maintain an audit trail consistently every time, no matter if it's a candidate updating their profile or a researcher adding a new trial.

3. Key Security Advantage

The primary security concern is protecting PHI. With this architecture, that sensitive data and the trial protocols are locked away in the Data Layer. The user-facing portals in the Presentation Layer never talk to the database directly. Any request for data has to go through the Matching Engine's DataAccess interface. This drastically reduces the number of potential attack vectors. It lets us focus all our security efforts—encryption, access control, and audit logging—on a single, well-protected boundary instead of sprinkling those responsibilities throughout all the UI components.

4. Key Performance Benefit

Matching Engine and the Notification Service is kept in the same Business Layer. This means they can communicate internally, like code calling code. We've avoided the overhead of having them as separate microservices that would need to talk to each other over the network with all that serialization and latency. This directly benefits the user experience. The candidate sees match results faster, and researchers get recruitment updates promptly, which is a key requirement for the near-real-time feel we're aiming for.